The Readers and Writers Problem

```
                    typedef int semaphore;            /* use your imagination */
                    semaphore mutex = 1;              /* controls access to 'rc' */
                    semaphore db = 1;                 /* controls access to the database */
                    int rc = 0;                       /* # of processes reading or wanting to */

                    void reader(void)
                    {
                         while (TRUE) {               /* repeat forever */
enterRegion(){               down(&mutex);            /* get exclusive access to 'rc' */
                             rc = rc + 1;             /* one reader more now */
                             if (rc == 1) down(&db);  /* if this is the first reader ... */
                             up(&mutex);              /* release exclusive access to 'rc' */
                             read_data_base( );       /* access the data */
                             down(&mutex);            /* get exclusive access to 'rc' */
 leaveRegion(){              rc = rc − 1;             /* one reader fewer now */
                             if (rc == 0) up(&db);    /* if this is the last reader ... */
                             up(&mutex);              /* release exclusive access to 'rc' */
                             use_data_read( );        /* noncritical region */
                         }
                    }


                    void writer(void)
                    {
                         while (TRUE) {               /* repeat forever */
                             think_up_data( );        /* noncritical region */
                             down(&db);               /* get exclusive access */
                             write_data_base( );      /* update the data */
                             up(&db);                 /* release exclusive access */
                         }
                    }
```

פתרון בעייתי
הקוראים – כותבים
יש הרעבה.
לכותבים

A solution to the readers and writers problem

פתרון בעיית הקוראים כותבים עם העדפה לכותבים

```
Shared variables:
    counter_protection_semaphore: Semaphore(1)  // Semaphore for mutual
    exclusion
    readers_writers_semaphore: Semaphore(1)  // Semaphore for controlling
    access to the resource
    readers_count: integer := 0  // Number of active readers
    writers_count: integer := 0  // Number of writers waiting to access the
    resource

Reader:
    loop:
        down(counter_protection_semaphore)
        readers_count += 1
        if readers_count == 1 and writers_count > 0:
            // If this is the first reader and there are writers waiting,
            // block other readers from starting
            down(readers_writers_semaphore)
        up(counter_protection_semaphore)

        // Start reading
        read_resource()

        down(counter_protection_semaphore)
        readers_count -= 1
        if readers_count == 0 and writers_count > 0:
            // If there are no more readers and there are writers waiting,
            // allow a writer to access the resource
            up(readers_writers_semaphore)
        up(counter_protection_semaphore)

Writer:
    loop:
        down(counter_protection_semaphore)
        if readers_count > 0 or writers_count > 0:
            // If there are active readers or other writers are waiting,
            // block this writer
            writers_count += 1
            up(counter_protection_semaphore)
            down(readers_writers_semaphore)
            down(counter_protection_semaphore)
            writers_count -= 1
            up(counter_protection_semaphore)
        else:
            up(counter_protection_semaphore)
            down(readers_writers_semaphore)

        // Start writing
        write_resource()

        // Release the resource
        up(readers_writers_semaphore)
```

משתמשים באותם משתנים וסמפורים מלבד לכותבים
המספר שמציין את הכמות של הכותבים שמחכים עדיין

לפני הקריאה

חוסמים קוראים נוספים
כדי שהבא בתור יהיה
לו הכותב

הקריאה

אחרי הקריאה

אם אין קוראים
ויש כותב שממתין אז
נשחרר סמפור לכותב

לפני כתיבה

אם יש כותב או קורא
פעילים - הכותב נכנס
להמתנה ומגדיל את המונה.

אם אין כותבים
או קוראים שממתינים
- עובר לכתוב

הכתיבה

אחרי כתיבה

מה שונה בקוד זה הוא שימוש לפתרון בעדיפות הקוראים.

בקוראים בודקים אם יש כותבים ממתינים ואם כן לא מאפשרים
להם להכנס וממתינים עם הקריאה.

בכותבים בודקים אם יש קורא או כותב אחר פעילים.
אם כן אז נכנסים להמתנה ומגדילים את מספר הכותבים.
אם אין אז חוסמים קוראים נוספים וממשיכים כרגיל.

הסמפור שמגן על מספר הקוראים מגן גם על מונה מספר
הכותבים.

מועד א

86 - [illegible]

חלק א (55 נקודות)

ענו על השאלות 1-3.

שאלה 1 (22 נקודות)

שני הסעיפים מתייחסים לבעיה הבאה.

במערכת ישנם שלושה תהליכונים (THREADS) A, B ו-C. יש לכתוב אלגוריתם (בפסאודוקוד או דמוי C) המממש קטע קריטי בעל התכונות הבאות:

אם A - נמצא בקטע הקריטי, אזי הוא נמצא בו לבדו.

תהליכונים B ו-C יכולים להימצא בקטע הקריטי בו זמנית. בפרט, אם אחד מהם נמצא בקטע הקריטי הוא אזי השני יכול להיכנס לקטע הקריטי גם כן, אפילו אם A כבר ממתין להיכנס לקטע קריטי.

על האלגוריתם לקיים את כל הדרישות של פתרון בעיית קטע קריטי, חוץ מהוגנות היא (FAIRNESS).

(13 נקי) א. ממשו את הפרוצדורות:

A-enter ,A-exit, B-enter , B-exit, C-enter ו- C-exit

המממשות את קטעי הכניסה והיציאה של התהליכונים לקטע קריטי בצורה היעילה ביותר.

בסעיף זה מותר להשתמש אך ורק במשתנים הבאים:

1. סמפורים S1 ו-S2 המאותחלים שניהם לערך 1.
2. משתנה C שמאותחל לערך 0.

יש כאן בעיה בניסוח של השאלה. C — זה לא משתנה כי צריך כאן להשתמש ב-counter.

אין להשתמש בהמתנה פעילה(busy waiting). כלומר, תהליכונים יכולים להמתין רק ע״י שינה על סמפור.

אין חובה להשתמש בכל המשתנים.

מבקשים מאיתנו לעשות לכתוב את הפתרון לבעיית הקוראים-כותבים עם סמפורים כאשר A כותב ו-B,C קוראים.

S1 - מגן על ה-counter של מספר הקוראים.

S2 - מסנכרן הקוראים וכותבים.

A [illegible] מסנכרן את S2

```
A-enter(){down(S2);}    B-enter(){ down(S1);
A-exit(){up(S2);}                  count++;
                                   if(c==1){
                                       down(S2);
                                   }
                                   up(S1);
                                }
B-exit(){down(S1);
         count--;
         if(c==0){
             up(S2);
         }
         up(S1);
}
```

לא לכתוב אם יש קוראים

ניתן לעשות [illegible] ה- [illegible] count מגן

אסור לכתוב

(9 נק') ב. גם בסעיף זה עליכם לממש את הפרוצדורות:

A-enter ,A-exit, B-enter , B-exit, C-enter ו C-exit

אבל בסעיף זה מותר להשתמש אך ורק במשתנים הבאים: m1, m2 ו-c שמאותחלים לאפס. תהליכונים הנאלצים להמתין יעשו זאת באמצעות busy waiting בעזרת פקודה test-and-set.

להזכירכם פעולה test-and-set(c,1) מתבצעת באופן אטומי, כותבת ערך 1 למשתנה c ומחזירה את ערכו הקודם שלו. אין להשתמש בסמפורים בסעיף הזה.

יש כאן 3 פונק' C שבכל פעם נכנס אחד:
פונקציה, המשתנה count ומשנה (מעלה) שמעדכנים את ערכו.

מבחינת זמן בריצה את אותו הפתרון כמו הסעיף הקודם רק שהפעם במקום סמפורים משתמש במנעולים.
ה-TSL עכשיו נועל את המנעול ומציב את ערכו הקודם- לכן אם מחזיר שהוא היה נעול עוד נלכד כך אז לא עושים כלום וכלום המנעול תפוס. אם היה פתוח סימן שעכשיו אפשר להיכנס לעבוד וגם נעלנו את המנעול.
m1 - מגן על ה-counter
m2 - מסנכרן בין הקורא לכותבים.

```
A-enter(){while(TSL(m2,1));}

A-exit(){m2=0;}

B-exit(){while(TSL(m1,1));
        count--;
        if(c==0){
          while(TSL(m2,1));
        }
        m1=0;
}
```

```
B-enter(){while(TSL(m1,1));
        count++;
        if(c==1){
          m2=0;
        }
        m1=0;
}
```

נעל את m2

17
(1)
8+9

צריך מימוש כמו קוראים כותבים

שאלון 521

86.52.3 M1

2022א - 88

חלק א (55 נקודות)

עליכם לענות על שלוש השאלות 1-3.

שאלה 1 (20 נקודות)

להלן פסודו-קוד המתאר את האלגוריתם לפתרון בעיית קוראים-כותבים ששונה מהנלמד בקורס.

בעיה

סימונים:

r מציין מונה מספר הקוראים.

sRead סמפור בינארי שמגן על מונה מספר הקוראים.

sWriteRead סמפור בינארי שמשמש למניעה הדדית בין קוראים לבין כותבים(ובין כותבים לעצמם). כמו אותו דבר כמו בבעיה הקודמת.

כל המשתנים הם גלובליים.

נתון:

```
int r = 0
semaphore sRead = 1
semaphore sWriteRead = 1
```

אותו דבר כמו פתרון ישיר — אותו דבר כמו פתרון ישיר

קוד של קורא	קוד של כותב

`down(sRead)`
`r++`
`if(r==1) then down(sWriteRead)`
`up(sRead)`
` // make some read job`
`down(sRead)`
`r--`
`if(r==0) then up(sWriteRead)`
`up(sRead)`
	`down(sWriteRead)`
	` // make some write job`
	`up(sWriteRead)`

(4 נק') ✓ א. האם האלגוריתם הזה נותן העדפה לקוראים או לכותבים או שהוא הוגן לשניהם באותה המידה(אם יש אותו מספר של קוראים וכותבים, למי יש בממוצע יותר סיכוי להתחיל)? הסבירו למה.

✓

תשובה והסבר:

יש 2 פתרונות לבעיית הקוראים-כותבים שהוצגו. הפתרון שלנו מופע במצגות של ויקי וזה לא נותן עדיפות לכותב. הפתרון שנותן עדיפות לכותבים מופע בעמוד 280.

הפתרון הזה נותן עדיפות לקוראים מפני שלא ניתן לכתוב כל עוד r≠0-כלומר כל עוד יש קוראים אז הם יבצעו הקוד.

נושא לא בא

נא לא לכתוב בשוליים

(3 נק׳) ✓ב. ידוע שבהרצה מסוימת כותב התחיל כתיבה לפני שקורא כלשהו התחיל את הקריאה.
כמו כן ידוע שיש עוד כותב שמחכה בניסיון להתחיל לכתוב ו-3 קוראים שמחכים בניסיון להתחיל לקרוא.
האם אחרי שהכותב שכבר כותב יסיים את הכתיבה מובטח שקורא כלשהו יתחיל את הקריאה לפני שכותב נוסף יתחיל את הכתיבה?

תשובה והסבר:

התשובה בסעיף הקודם לא תשתנה - היא אומרת שכל עוד
אנחנו קוראים אז נשאר בקריאה אבל כן אנחנו לא יודעים
עדין אלו אחרי כתיבה. במקום הסמפור שמשתמשות למה.
וזה תלוי שהמתזמן יבחר יכנס - וזה תלוי של הוא ויש
מזל.

(6 נק׳) ✓ג. הוצע שינוי הבא לאלגוריתם הנ״ל שנראה ככה:

קוד של קורא

```
down(sRead)
r++
up(sRead)
if (r==1) then down(sWriteRead)
 // make some read job
down(sRead)
r--
if (r==0) then up(sWriteRead)
up(sRead)
```

קוד של כותב

```
down(sWriteRead)
 // make some write job
up(sWriteRead)
```

האם האלגוריתם החדש עומד בכל הדרישות של מניעה הדדית? הוכיח שהאלגוריתם עומד בכל הדרישות של מניעה הדדית או הפריכו ע״י דוגמא נגדית וציינו אלו דרישות מופרות אם לא עומד.

✓

תיקון והסבר:

השינוי כאן הוא מה שמופיע בין שורה 3 ל-4. לדעתי
הבעיה היא r==1 מבוצע מחוץ לקטע קריטי ולכן
יכול להיות מצב שבו 2 קוראים מבצעים את 3 השורות
הראשונות ואז r=2 ו- אז בשורה 4 לא נכנס.
ולכן שבמצב שיש קוראים יכול להכנס כותב נוסף.
לכן אין מניעה הדדית.

6

נא לא לכתוב בשוליים

(7 נקי) ✓ד. הוצע שינוי אחר לאלגוריתם הנ"ל שנראה ככה:

קוד של קורא	קוד של כותב

`down (sRead)`
`r++`
`if (r==1) then down(sWriteRead)`
`up(sRead)`
` // make some read job` ← r=1
`down(sRead)`
`r–`
`up(sRead)` ← r=1
`if (r==0) then up(sWriteRead)`
	`down (sWriteRead)`
	` // make some write job`
	`up(sWriteRead)`

האם האלגוריתם החדש עומד בכל הדרישות של מניעה הדדית? הוכיח שהאלגוריתם עומד בכל הדרישות של מניעה הדדית או הפריחו ע"י דוגמא נגדית וציינו אלו דרישות מופרות אם לא עומד בהן.

תשובה והסבר:

השינוי כאן הוא בין השורה האחרונה לפני האחרונה.
זה לא תקין כי אם r=0 מבצעת עוד ללא עד r.
יכול להיות מצב שבו מבצעים 2 קוראים חוץ מזה
האחרונה ועוברים לשלב זה מסוף לקוראים שאין עוד
קוראים. אפילו אם יכנס קורא נוסף הוא יחכה בקטע
הקריטי מכיוון שהסמפור sWriteRead כבר נלקח.

נועם לנדא

נא לא לכתוב בשוליים